

P3409R0: Enabling more efficient stop-token based cancellation of senders

Table of Contents

- [1. Abstract](#)
- [2. Overview](#)
- [3. Motivation](#)
 - [3.1. Why does it need to be done now?](#)
- [4. Proposal](#)
 - [4.1. Allowing stoppable token that only support a finite number of stop-callbacks](#)
 - [4.2. Limiting when stop-callbacks can be constructed by sender algorithms](#)
 - [4.3. Adding the `std::single\_inplace\_stop\_source` type](#)
 - [4.4. Adding the `std::finite\_inplace\_stop\_source<N>` class template](#)
 - [4.4.1. Tweaks to the stoppable token and stoppable-source concepts](#)
 - [4.5. Modifications to `std::execution` sender algorithms](#)
 - [4.5.1. Changes to split algorithm](#)
 - [4.5.2. Changes to when all algorithms](#)
 - [4.5.3. Alternative: Leave the choice of stop-token to be implementation-defined](#)
- [5. Design Discussion](#)
 - [5.1. Performance Benefits](#)
 - [5.1.1. Cost of inplace stop token](#)
 - [5.1.2. Cost of single inplace stop token](#)
 - [5.1.3. Cost of finite inplace stop token](#)
 - [5.2. Performance vs Safety Tradeoff](#)
 - [5.3. Usage in task coroutines](#)
 - [5.4. Do we still need inplace stop token?](#)
- [6. Proposed Wording](#)
- [7. Appendix A - Benchmarks](#)
 - [7.1. Register/unregister stop-callbacks single-threaded](#)
 - [7.2. Call request_stop\(\) with no callbacks](#)
 - [7.3. Call request_stop\(\) with x/y callbacks](#)
 - [7.4. Register/unregister callbacks from two threads concurrently](#)
- [8. Appendix B - Implementation of single_inplace_stop_source](#)

Document **P3409R0**

Date **2024-10-16**

Reply To **Lewis Baker <lewissbaker@gmail.com>**

Audience **SG1**

1. Abstract

This paper proposes placing semantic constraints on consumers of a stop-token obtained from the `std::get_stop_token` query on a receiver to allow more efficient implementations of the `stoppable_token` data-structure to be used in sender-algorithms where there are a finite number of child operations.

It also proposes adding a new `std::single_inplace_stop_source` type that implements such a data-structure that only allows a single stop-callback to be registered at a time.

It also proposes adding a new `std::finite_inplace_stop_source<N>` class template that is similar to an array of `N` `std::single_inplace_stop_source` objects in that it can allow up to `N` consumers but is more space and cpu-efficient than such an array.

Finally, it proposes updating the `split()` algorithm to use `std::single_inplace_stop_source` and the `when_all()` algorithm to use `std::finite_inplace_stop_source` instead of `std::inplace_stop_source` as currently specified.

This paper does not propose removing/replacing `std::inplace_stop_token` as there are still use-cases for a stop-token that can support registration of an unbounded number of stop-callbacks.

This paper is one of two alternatives to be considered for improving the performance of cancellation of sender-based asynchronous operations. The other paper to be considered along-side this paper is [P3410](#) "Enabling more efficient sender cancellation without stop-tokens", which explores a design that puts a `request_stop()` method on the operation-state objects instead of using stop-tokens to communicate stop-requests.

2. Overview

The design proposed for sender/receiver-based asynchronous operations in P2300R10 included facilities for cooperatively cancelling in-flight operations and is based on a generalisation of the `std::stop_token` design added in C++20.

The current design of `std::stoppable_token` and the `std::get_stop_token` query on a receiver's environment permits consumers of the stop-token to attach an arbitrary number of stop-callback objects to a given stop-token obtained from a receiver's environment.

This means that implementations of the `std::stoppable_token` concept, such as `std::stop_token` and `std::inplace_stop_token`, are required to use a data-structure that supports registration of an arbitrary number of stop-callbacks. This usually involves a doubly-linked intrusive list with some form of mutex (usually a spin-mutex) to synchronize access to the stop-callback list.

However, there are some cases where an algorithm has a finite number of child operations and where each child operation generally only needs to register a single stop-callback.

Implementations of the `stoppable_token` concept can make use of simpler and more efficient data-structures if it can assume that there are a finite number of consumers of the stop-token, each of which will only ever register a single stop-callback.

This paper proposes enabling these simpler data-structures by modifying the semantic constraints of stop-tokens returned from `std::get_stop_token()` on a receiver's environment to limit usage of the stop-token to register at most one stop-callback unless otherwise specified.

This paper also proposes adding two new families of classes implementing the `stoppable_token` concept which take advantage of these semantic constraints on usage:

- `std::single_inplace_stop_source` - Supports a single associated stop-callback.
- `std::finite_inplace_stop_source<N>` - Equivalent to a tuple of `N` separate `std::single_inplace_stop_source` objects, providing `N` independent stop-tokens, each of which supports a single associated stop-callback.

3. Motivation

When the paper P2175R0 "Composable cancellation for sender-based async operations" was reviewed by SG1, concerns were raised about the potential performance cost of the stop-token-based design of the cancellation mechanisms due to the amount of synchronization involved.

In order to be able to cancel some kinds of asynchronous operations after they have started, the operation may need to register a stop-callback that will execute some logic to interrupt the operation in the event that a stop-request is made before the operation completes. For example, such logic might removing an entry from a queue of scheduled tasks, or might call an underlying OS API to cancel a pending I/O operation.

Before the operation completes and the operation-state is destroyed, the operation then needs to unregister the callback.

In cases where there are many leaf operations, each of which need to register and unregister a stop-callback, and where the operations themselves may complete quickly much of the time, and where a stop-request being made is relatively rare, the overhead of constantly registering and deregistering the stop-callback can potentially be significant.

For example, when repeatedly reading from a socket where the socket constantly has buffered data available, we might issue an async recv operation, register a stop-callback just in case the operation takes a long time and a stop-request were to be issued, only to have the operation completion almost immediately, then requiring us to deregister the stop-callback before producing the result for the consumer.

With the current design of `std::inplace_stop_token`, the registration/deregistration of stop-callbacks typically involves acquiring a spin-lock on the data-structure, updating a doubly-linked list of registered stop-callbacks and then releasing the spin-lock.

Whilst `std::inplace_stop_token` is still a relatively lightweight data-structure - it doesn't do an dynamic allocation, like `std::stop_token` does - we would like to try and minimise the overhead needed for registering and deregistering stop-callbacks where possible to improve the performance of situations like the one described above.

Allowing the use of a simpler data-structure in these cases would reduce the amount of synchronization involved (although there is still *some* synchronization required), reduce the amount of book-keeping required, and also reduce the size needed within operation-states for storing stop-callbacks.

3.1. Why does it need to be done now?

If we do not apply this change before releasing sender/receiver, we will not be able to apply this optimization later.

If the status quo is kept then users may write their own sender algorithm implementations that take advantage of the fact that they can register multiple stop-callbacks. e.g. by passing an inherited stop-token to multiple child operations which run concurrently and each register a stop-callback.

Adding a restriction on how stop-tokens can be used later would be a breaking change as trying to compose such user-defined algorithms with algorithms that wanted to take advantage of the stop-token restrictions would lead to undefined behaviour due to that user-code potentially trying to register multiple stop-callbacks associated with the stop-token, resulting in some pre-condition violations.

Also, trying to change an algorithm implementation from using `std::inplace_stop_source` to later using `std::single_inplace_stop_source` is going to change the layout of operation-state types and would thus be a potential ABI break for that algorithm.

4. Proposal

This paper proposes several changes:

- Allowing `std::get_stop_token()` to return a `stoppable_token` type that only supports one associated stop-callback to exist at a time.
- Defining a `std::single_inplace_stop_source` type that only supports a single associated stop-callback to exist at a time.
- Defining a `std::finite_inplace_stop_source<N>` type that provides N separate stop-tokens, each of which supports a single associated stop-callback, but where sending a stop-request via the stop-source sends the stop-request to all N stop-tokens.
- Modifying the `split()` algorithm to use `std::single_inplace_stop_source` instead of `std::inplace_stop_source`.
- Modifying the `when_all()` algorithm to use `std::finite_inplace_stop_source` instead of `std::inplace_stop_source`.

4.1. Allowing `stoppable_token` that only support a finite number of stop-callbacks

There are no syntactic changes required to the `stoppable_token` concept in order to support this.

However, to make the intent clear, we need to add a paragraph to the description of stop-tokens that states that for a given type that models *stop-callback-for*, its constructor may have a pre-condition that the number of associated stop-callback objects is less than some positive, finite number.

This explicitly grants permission to implementers of the concept to add such a pre-condition to its stop-callback constructor. It also means that consumers of a generic stop-token must assume that the stop-callback constructor may have such a pre-condition, potentially with a maximum number of existing associated stop-callback objects that is zero, and therefore such generic consumers should limit themselves to constructing a single stop-callback object associated with the stop-token.

Implementations of stop-callback types are still free to define their constructor without such a precondition, and it is still valid for consumers of the corresponding stop-token type to construct multiple associated stop-callback objects.

For example, if I write a function that takes a `std::inplace_stop_token` then I know that this type allows an unbounded number of associated `std::inplace_stop_callback` objects and so, within the function I can safely construct multiple associated stop-callback objects.

```
void func1(std::inplace_stop_token st) {
    const auto on_stop = [] { /* do something */ };

    // Constructing multiple stop-callbacks is allowed.
    std::inplace_stop_callback cb1{st, [&] noexcept { /* do something */ }};
    std::inplace_stop_callback cb2{st, [&] noexcept { /* do something else */ }};

    // ...
}
```

However, if I were to write a function-template that could be instantiated with any type that satisfied `std::stoppable_token`, then I would need to limit myself to constructing at most one associated stop-callback object at a time.

```
template<std::stoppable_token StopToken>
void func2(StopToken st) {
    const auto on_stop = [] { /* do something */ };
    using callback_t = std::stop_callback_for_t<StopToken, decltype(on_stop)>;
```

```
// Constructing a single stop-callback is OK
callback_t cb1{st, on_stop};

// Constructing a second stop-callback would potentially be a pre-condition
// violation if StopToken happens to be e.g. std::single_inplace_stop_token.
callback_t cb2{st, on_stop};
}
```

Further, I would also be implying that my function also has a pre-condition that my caller provide me with a stop-token that permits me to construct at least one associated stop-callback. This would prevent them from passing, for example, a `std::single_inplace_stop_token` that already had an associated stop-callback object.

```
template<std::stoppable_token StopToken>
void func3(StopToken st) {
    const auto on_stop = [] { /* do something */ };
    using callback_t = std::stop_callback_for_t<StopToken, decltype(on_stop)>;

    callback_t cb{st, on_stop};

    // ...
}

void caller() {
    std::single_inplace_stop_source ss;

    func3(ss.get_token()); // OK. stop-token allows constructing a stop-callback

    std::single_inplace_stop_callback cb{ss.get_token(), [] { /* do something */ }};

    func3(ss.get_token()); // BUG: violates func3's pre-condition.
                          // stop-token already has an associated stop-callback.
}
```

4.2. Limiting when stop-callbacks can be constructed by sender algorithms

One of the use-cases that needs to be carefully considered here are algorithms, like `schedule_from`, which are specified to connect multiple child operations ahead of time, but only actually executes one of the child operations at a time.

These are algorithms where `connect()` on the parent operation calls `connect()` on two (or more) child operations, but where `start()` on the parent operation calls `start()` on the first child but `start()` on the second child is not called until after the completion of the first child. i.e. where the execution of the child operations does not overlap in time.

Consider the case where such a parent operation is provided an environment with a stop-token that only permits a single stop-callback (such as the proposed `std::single_inplace_stop_token`).

It would be preferable to allow passing through this stop-token to both children rather than having to construct a separate `std::finite_inplace_stop_source<2>` and provide different stop-tokens to each child and then also attach a stop-callback to the provided stop-token which forwards stop-requests through to a call to `request_stop()` on the local stop-source.

However, in order to guarantee that we do not violate the pre-conditions of the stop-callback constructor, we need to ensure that the child operations do not both attempt to construct stop-callbacks with overlapping lifetimes.

The current wording for [exec.recv.concepts] p3 states:

Let `rcvr` be a receiver and let `op_state` be an operation state associated with an asynchronous operation created by connecting `rcvr` with a sender. Let `token` be a stop token equal to `get_stop_token(get_env(rcvr))`. `token` shall remain valid for the duration of the asynchronous operation's lifetime ([`exec.async.ops`]).

[Note: This means that, unless it knows about further guarantees provided by the type of `rcvr`, the implementation of `op_state` cannot use `token` after it executes a completion operation. This also implies that any stop callbacks registered on `token` must be destroyed before the invocation of the completion operation. — end note]

This references [`exec.async.ops`] p7 which defines "asynchronous operation lifetime":

The lifetime of an asynchronous operation, also known as the *operation's async lifetime*, begins when its start operation begins executing and ends when its completion operation begins executing. If the lifetime of an asynchronous operation's associated operation state ends before the lifetime of the asynchronous operation, the behavior is undefined. After an asynchronous operation executes a completion operation, its associated operation state is invalid. Accessing any part of an invalid operation state is undefined behavior.

The important parts of these two paragraphs are that the stop-token obtained from `get_stop_token(get_env(rcvr))` is only required to be valid for the duration of the asynchronous operation's lifetime, and that an asynchronous operation's lifetime starts at the beginning of the call to `start()` on the operation-state and ends at the beginning of the call to a completion-function.

This implies that, unless you have additional information about the validity of a stop-token provided in the environment, you should not assume that it is valid to construct a stop-callback associated with that stop-token (or indeed do anything else you can't do with an invalid stop-token) until the `start()` operation on the operation-state is called.

This constraint placed on sender algorithms and their use of stop-tokens should be sufficient to guarantee that it is safe for the class of algorithms discussed above, for example `schedule_from`, to pass through a `single_inplace_stop_token` from its environment to the environment passed to child operations.

The one change I would suggest here is to explicitly call out this restriction in the note, similarly to how the note calls out that stop-callbacks must be destroyed before the invocation of the completion operation. In particular it should call out that stop-callbacks should not be constructed until after the beginning of the invocation of the `start()` method on the operation-state.

A parent operation only needs to introduce a new stop-source and give separate stop-tokens to child operations if both of the following are true:

- we don't know that the stop-token can support multiple stop-callbacks at the same time; and
- the child operations have overlapping asynchronous operation lifetimes; and
- the parent operation wants to forward stop-requests to child operations

4.3. Adding the `std::single_inplace_stop_source` type

Proposes adding the following class and class-template definitions the `<stop_token>` header:

```
namespace std
{
    class single_inplace_stop_token;
    template <std::invocable CB>
    class single_inplace_stop_callback;
```

```

class single_inplace_stop_source {
public:
    single_inplace_stop_source() noexcept;
    ~single_inplace_stop_source();

    single_inplace_stop_source(const single_inplace_stop_source&) = delete;
    single_inplace_stop_source(single_inplace_stop_source&&) = delete;
    single_inplace_stop_source& operator=(const single_inplace_stop_source&) = delete;
    single_inplace_stop_source& operator=(single_inplace_stop_source&&) = delete;

    bool stop_possible() const noexcept;
    bool stop_requested() const noexcept;

    bool request_stop() noexcept;

    single_inplace_stop_token get_token() const noexcept;
};

class single_inplace_stop_token {
public:
    template <typename CB>
    using callback_type = single_inplace_stop_callback<CB>;

    single_inplace_stop_token() noexcept;
    single_inplace_stop_token(const single_inplace_stop_token&) noexcept;
    single_inplace_stop_token(single_inplace_stop_token&&) noexcept;
    ~single_inplace_stop_token();

    single_inplace_stop_token& operator=(const single_inplace_stop_token&) noexcept;
    single_inplace_stop_token& operator=(single_inplace_stop_token&&) noexcept;

    bool stop_possible() const noexcept;
    bool stop_requested() const noexcept;

    friend bool operator==(const single_inplace_stop_token& a,
                           const single_inplace_stop_token& b) noexcept;

private:
    single_inplace_stop_source* source; // exposition only
};

template <std::invocable CB>
class single_inplace_stop_callback {
public:
    template <typename Initializer>
    requires std::constructible_from<CB, Initializer>
    single_inplace_stop_callback(single_inplace_stop_token st,
                                Initializer&& init)
        noexcept(std::is_nothrow_constructible_v<CB, Initializer>);

    ~single_inplace_stop_callback();

    single_inplace_stop_callback(const single_inplace_stop_callback&) = delete;
    single_inplace_stop_callback(single_inplace_stop_callback&&) = delete;
    single_inplace_stop_callback& operator=(const single_inplace_stop_callback&) = delete;
    single_inplace_stop_callback& operator=(single_inplace_stop_callback&&) = delete;

private:
    single_inplace_stop_source* source; // exposition only
    CB cb; // exposition only
};

template <typename CB>
single_inplace_stop_callback(single_inplace_stop_token, CB)

```

```
-> single_inplace_stop_callback<CB>;  
}
```

The semantics of these types are identical to that of the corresponding `std::inplace_stop_token`, `std::inplace_stop_source` and `std::inplace_stop_callback<CB>` types, with the exception that the `std::single_inplace_stop_callback` constructor has a pre-condition that there are no other stop-callback objects associated with the `std::single_inplace_stop_token` object passed to the constructor.

4.4. Adding the `std::finite_inplace_stop_source<N>` class template

In cases where a sender algorithm has multiple child operations, where the number of child operations is statically known, and where the algorithm wants to be able to communicate a stop-request to all of the child operations, using an array of `std::single_inplace_stop_source` objects is, in most cases, still going to be more efficient than using a `std::inplace_stop_source`.

However, naively storing an array of `std::single_inplace_stop_source` objects still has some overheads due to redundancy in the data-structures in the case where a stop-request is communicated to all of the stop-sources at the same time (sequentially on the same thread).

The `std::single_inplace_stop_source` data-structure contains an atomic pointer and also an atomic `std::thread::id` which is used to determine whether stop-callback deregistration is occurring from inside a call to `request_stop()`.

If we store an array of `N` `std::single_inplace_stop_source` objects, then we are storing `N` copies of this `std::thread::id` value, even though in this case, they will all contain the same value. We can save some storage in this case, by instead defining a data-structure that has `N` atomic pointers but only one atomic `std::thread::id` value.

Such a data-structure would have identical performance and layout to `std::single_inplace_stop_source` for `N=1`, but would save `(N-1)` pointers of storage for `N>=2`.

This paper proposes adding an implementation of such a data-structure, named `std::finite_inplace_stop_source`, which is templated on the desired number of independent stop-tokens that need to be supported.

The synopsis for this class-template is as follows:

```
namespace std  
{  
    template <size_t N, size_t Idx>  
    class finite_inplace_stop_token;  
    template <size_t N, size_t Idx, std::invocable CB>  
    class finite_inplace_stop_callback;  
  
    template <size_t N>  
    class finite_inplace_stop_source {  
    public:  
        finite_inplace_stop_source() noexcept;  
        ~finite_inplace_stop_source();  
  
        finite_inplace_stop_source(const finite_inplace_stop_source&) = delete;  
        finite_inplace_stop_source(finite_inplace_stop_source&&) = delete;  
        finite_inplace_stop_source& operator=(const finite_inplace_stop_source&) = delete;  
        finite_inplace_stop_source& operator=(finite_inplace_stop_source&&) = delete;  
  
        bool stop_possible() const noexcept;  
        bool stop_requested() const noexcept;
```



```

    bool request_stop() noexcept;

    template <size_t Idx>
        requires (Idx < N)
    finite_inplace_stop_token<N, Idx> get_token() const noexcept;
};

template <size_t N, size_t Idx>
class finite_inplace_stop_token {
public:
    template <typename CB>
        using callback_type = finite_inplace_stop_callback<N, Idx, CB>;

    finite_inplace_stop_token() noexcept;

    bool stop_possible() const noexcept;
    bool stop_requested() const noexcept;

    friend bool operator==(const finite_inplace_stop_token& a,
                           const finite_inplace_stop_token& b) noexcept;

private:
    finite_inplace_stop_source<N>* source_; // exposition-only
};

template <size_t N, size_t Idx, std::invocable CB>
class finite_inplace_stop_callback {
public:
    template <typename Init>
        requires std::constructible_from<CB, Init>
    finite_inplace_stop_callback(
        finite_inplace_stop_token<N, Idx> st,
        Init&& init) noexcept(std::is_nothrow_constructible_v<CB, Init>);

    ~finite_inplace_stop_callback();

    finite_inplace_stop_callback(const finite_inplace_stop_callback&) = delete;
    finite_inplace_stop_callback(finite_inplace_stop_callback&&) = delete;
    finite_inplace_stop_callback& operator=(const finite_inplace_stop_callback&) = delete;
    finite_inplace_stop_callback& operator=(finite_inplace_stop_callback&&) = delete;

private:
    CB cb; // exposition-only
    finite_inplace_stop_source<N>* source_; // exposition-only
};

template <size_t N, size_t Idx, typename CB>
finite_inplace_stop_callback(finite_inplace_stop_token<N, Idx>, CB)
    -> finite_inplace_stop_callback<N, Idx, CB>;
}

```

An instance of `finite_inplace_stop_source<N>` has `N` separate associated `finite_inplace_stop_token<N, Idx>` stop-tokens, where `Idx` is in the range `0 .. N-1`.

Each `finite_inplace_stop_token<N, Idx>` from a given stop-source can have at most one associated `finite_inplace_stop_callback<N, Idx>` object at a time.

When a call to `request_stop()` is made on the stop-source object, the stop-request is sent to all of the associated stop-tokens. Further, any stop-callbacks associated with any of the associated stop-tokens will be invoked.

The intent here is that it would be a valid implementation of `finite_inplace_stop_source<N>` to just hold an array of `single_inplace_stop_source` objects and to have `request_stop()` forward to a call to `request_stop()` on all of the `single_inplace_stop_source` objects. However, a high QoI implementation may choose to use a more efficient data-structure.

There is also the question of whether we should permit this class to be instantiated with a template-parameter of 0 or not. i.e. is it valid to write `finite_inplace_stop_source<0>`.

Such an object would not have the ability to obtain any associated stop-tokens and therefore would not have the ability to register any stop-callbacks. Ideally, such a type would compile out to nothing.

To enable this optimization, the `finite_inplace_stop_source<N>::stop_possible()` method returns `N >= 1`. This means that it will return `false` for `N == 0`, a case when there is no possibility of obtaining an associated-stop token that could observe a stop-request. Such a stop-source object is already possible with `std::stop_source(std::nostopstate)` and is called a *disengaged* stop-source.

This allows implementations to provide a specialization for `finite_inplace_stop_source<0>` that is an empty class, rather than having to have an `atomic_flag` data-member just to make sure that `request_stop()` returns `true` on first invocation and `false` on subsequent invocations.

For example, a possible implementation of this specialization may be:

```
namespace std
{
    template<>
    class finite_inplace_stop_source<0> {
    public:
        finite_inplace_stop_source() noexcept = default;

        finite_inplace_stop_source(const finite_inplace_stop_source&) = delete;
        finite_inplace_stop_source(finite_inplace_stop_source&&) = delete;
        finite_inplace_stop_source& operator=(const finite_inplace_stop_source&) = delete;
        finite_inplace_stop_source& operator=(finite_inplace_stop_source&&) = delete;

        bool stop_possible() const noexcept { return false; }
        bool stop_requested() const noexcept { return false; }
        bool request_stop() noexcept { return false; }
    };
}
```

4.4.1. Tweaks to the `stopbable_token` and `stopbable-source` concepts

The nature of the `finite_inplace_stop_source<N>` type is such that the existing definitions of `stopbable-source` and `stopbable_token` as described in [thread.stoptoken.intro] and [stoptoken.concepts] do not quite fit the type, yet the `finite_inplace_stop_source` family of types is something that I think the concepts should support.

1. Relaxing `stopbable-source`

The exposition-only `stopbable-source` concept definition currently requires that the type has a `get_token()` member-function that returns a `stopbable_token`. However, the `finite_inplace_stop_source<N>` type has a `get_token<Idx>()` member-function, and thus the `finite_inplace_stop_source<N>` type would not satisfy the `stopbable-source` concept.

This paper therefore proposes to remove this syntactic requirement from `stopbable-source` and to instead just provide a semantic requirement that there is *some* way to obtain a `stopbable_token` that is associated

with the stop-source.

2. Associated stop-callbacks

One of the other challenges with the current wording is that it refers to stop-tokens, stop-callbacks and stop-source objects that share a stop-state being "associated" with each other.

We have two choices with regards to how to apply this logic to the `finite_inplace_stop_source` family of types.

The first is to treat a `finite_inplace_stop_source<N>` object as having N separate stop-states, with each `finite_inplace_stop_token<N, Idx>` referring to a particular stop-state. The `request_stop()` method on the `finite_inplace_stop_source` has the semantics of sending a stop-request to each of the N stop-states.

This would make it easier to define the pre-condition necessary on the constructor of a `finite_inplace_stop_callback` object, as a stop-callback constructed using `source.get_token<0>()` would not be considered associated with a stop-callback constructed using `source.get_token<1>()`, since they would refer to different stop-states. We could just place a pre-condition on the stop-callback constructor that requires that there are no existing stop-callbacks associated with the stop-token.

However, if we take this approach, we might want to modify the definition such that a `stoppable-source` could potentially be associated with multiple stop-states. It is currently limited to being associated with at most one stop-state.

The second approach is to treat the `finite_inplace_stop_source<N>` as having a single stop-state such that all of the stop-callbacks registered using the different stop-token types are associated with the `finite_inplace_stop_source` object.

This would then require some other way of describing the pre-condition on the stop-callback construction. For example, we might need to define a pre-condition like "there are no stop-callbacks associated with the provided stop-token argument which have the same `Idx` template argument as the stop-callback currently being constructed."

This paper proposes the latter approach as a less-intrusive modification to the status-quo.

4.5. Modifications to `std::execution` sender algorithms

Of the initial set of sender/receiver algorithms added in P2300R10, there are two algorithms which are currently specified to construct their own `std::inplace_stop_source` object which could be replaced with `std::single_inplace_stop_source - std::execution::when_all()` and `std::execution::split()`.

Other than the types of stop-tokens returned from queries on the environments passed to child operations, there should be no changes in user-visible behaviour of these algorithms.

4.5.1. Changes to `split` algorithm

The `split` algorithm wraps a single child operation in a copyable sender that has shared ownership semantics of the wrapped child operation.

The shared-state is specified to include a stop-source object of type `std::inplace_stop_source` and the environment of the receiver connected to the wrapped sender returns an associated `std::inplace_stop_token` from the `std::get_stop_token` query.

This paper proposes to just change the specification for this stop-source object to be of type `std::single_inplace_stop_source` and for the environment passed to the child operation to return a

`std::single_inplace_stop_token` object from its `std::get_stop_token` query.

4.5.2. Changes to `when_all` algorithms

The default implementation of the `when_all` algorithm, and the `when_all_into_variant` algorithm by implication as it is defined in terms of `when_all`, are specified to have the operation-state owning a `std::inplace_stop_source` which is used to communicate a stop-request to all child operations of the `when_all` operation.

This paper proposes to replace the `std::inplace_stop_source` with an instance of `std::finite_inplace_stop_source<N>` where `N` is the number of child senders passed to the `when_all()` algorithm.

The environment of the receiver connected to `I`'th child operation would provide an environment from its `get_env()` method whose `std::get_stop_token` query returned the result of calling the `get_token<I>()` member-function on the stop-source object.

4.5.3. Alternative: Leave the choice of stop-token to be implementation-defined

An alternative approach that could be considered for these algorithms is to leave the stop-token type passed via the environment to child operations as unspecified and leave it up to implementers to choose the most appropriate stop-token/stop-source type.

If we decide that we don't want to add the new `single_inplace_stop_token` and `finite_inplace_stop_token` facilities to the standard library, but still want to apply the semantic constraints on `stoppable_token` then this approach could let implementations define their own internal stop-token types equivalent to the types proposed here and use them instead of `inplace_stop_token`.

However, if we decide to add the `single_inplace_stop_token` and `finite_inplace_stop_token` types into the standard library, then I don't see any significant downsides to specifying that the `split` and `when_all` algorithms are defined in terms of them.

5. Design Discussion

5.1. Performance Benefits

5.1.1. Cost of `inplace_stop_token`

The existing `inplace_stop_token` facility added by P2300R10 allows registering an arbitrary number of stop-callbacks without requiring any dynamic memory allocations through the use of an intrusive linked list of stop-callbacks.

The maintenance of the linked list of stop-callbacks requires synchronisation to ensure that multiple threads concurrently trying to register/deregister/invoke callbacks do not introduce data-races.

Here, we explore the cost of certain operations on typical implementations of this class so we can understand the potential savings by using a simpler data-structure.

I will be using the reference implementation from the [stdexec](#) library for this analysis.

1. Sizes of data-structure

The `inplace_stop_source` structure needs to store the following data members:

- `std::atomic<uint8_t>` - Synchronization state / flags.
- `stop_callback_base*` - A pointer to first item in linked list of registered callbacks.
- `std::thread::id` - thread-id of the thread that first called `request_stop()` This is needed to determine whether to block inside stop-callback deregistration or not.

If these data-members are appropriately laid out, then on 64-bit platforms this structure will usually be either 16 bytes or 24 bytes in size, depending on the size of your platform's `thread::id` type.

The `inplace_stop_callback` data-structure needs to store the following:

- pointer to the `inplace_stop_source` (so it can deregister itself)
- pointers to next/prev elements in the intrusive linked list
- a function-pointer for the callback to run
- synchronization state needed to signal when the callback has finished executing
- an additional pointer to a flag that is used to determine whether the stop-callback has been deregistered during the execution of the stop-callback
- The user-provided callback object itself

The net result is that the stop-callback object typically has a size of 6 pointers plus the size of the user's callback, which often itself contains a single pointer. So a total size of 7 pointers or 56 bytes.

2. Cost of operations

The `inplace_stop_token` class permits multiple threads to concurrently register/deregister callbacks. Therefore the registration/deregistration requires synchronization to ensure there are no data-races. As the synchronization operations are generally the most expensive part of registering a stop-callback, we will largely focus on the number of synchronization operations required.

1. Registering a stop-callback

The set of steps performed during construction of an `inplace_stop_callback` object is as follows:

- Tries to acquire a lock on the stop-source using a spin-lock
 - Enters a loop that performs a weak compare-exchange, trying to set the 'locked' flag from 0->1.
 - If it sees the 'locked' flag is already 1 then calls `wait()` on the atomic to allow the current thread to block efficiently until the synchronization state changes (hopefully setting the 'locked' flag to 0 or, alternatively, setting the 'stop-requested' flag to 1.
 - If it sees that there has already been a stop-request before the lock could be acquired then it abandons the attempt to lock and just invokes the stop-callback it was trying to register inline.
- Inserts the stop-callback object into the linked list
- Unlocks the spin-lock and notifies any threads that might be waiting to acquire the lock.

If the operation is uncontended then the best-case execution is:

- atomic load relaxed
- atomic compare-exchange weak acq-rel
- insert linked-list node
- atomic store release
- atomic notify

If the registration is contended and a thread is unable to acquire the lock immediately, then this becomes:

- atomic load relaxed

- repeat until successful
 - repeat until lock is available
 - atomic wait relaxed
 - atomic load relaxed
 - atomic compare exchange weak acq-rel
- insert linked-list node
- atomic store release
- atomic notify

It is worth noting that the use of a spin-lock here with a single locked-flag does not guarantee fairness among threads. A thread may spin in the above loops for an unbounded amount of time waiting to acquire the lock if the lock is highly contended.

Other strategies could potentially be used here (e.g. ticket-based locks) which could improve fairness, although at the cost of additional synchronization.

2. Deregistering a stop-callback

When a stop-callback object is destroyed, the stop-callback needs to be removed from the list of registered callbacks.

This requires acquiring a lock on the spin-lock, similar to that required during callback registration but without the early-out if a stop-request has been made.

However, it also needs to handle the case where the stop-callback has been run on another thread, in which case we need to wait until the other thread indicates the callback has finished executing.

In the case where a stop-request has not been made, and there is no contention, this operation performs:

- atomic load relaxed
- atomic compare-exchange weak acq-rel
- remove node from linked-list
- atomic store release
- atomic notify

In the case that a stop-request has not been made, but there is contention, the operation may perform:

- atomic load relaxed
- repeat until successful
 - repeat until 'locked' flag is not set
 - atomic wait relaxed
 - atomic load relaxed
 - atomic compare-exchange weak acq-rel
- remove node from linked-list
- atomic store release
- atomic notify

In the case that a stop-request has been made on another thread and the callback invocation has already completed, the operations will include:

- atomic load relaxed (read 'locked' flag = 0)
- atomic compare-exchange weak acq-rel (set 'locked' flag = 1)
- read node state - notice that callback has already been removed
- atomic store release (set 'locked' flag = 0)

- atomic notify
- atomic load acquire (read 'callback-completed' flag = 1)

If there has been a stop-request on another thread and the callback invocation has not yet returned then the operations will include:

- atomic load relaxed (read 'locked' flag = 0)
- atomic compare-exchange weak acq-rel (set 'locked' flag = 1)
- read node state - notice that callback has already been removed
- atomic store release (set 'locked' flag = 0)
- atomic notify
- atomic load acquire (read 'callback-completed' flag - read 0)
- atomic wait acquire (until 'callback-completed' flag is non-zero)

3. Calling request_stop()

The call to `inplace_stop_source::request_stop()` needs to first atomically acquire the lock and set the 'stop-requested' flag. If successful, then the calling thread is responsible for invoking the registered stop-callbacks. As the lock must not be held during the invocation of the callbacks to prevent potential deadlocks, the spin-lock must be released and reacquired for each registered callback.

Further, for each callback, as the deregistration of the callback can potentially be executing concurrently on another thread and become blocked waiting for the invocation of the callback to complete, the thread calling `request_stop()` must perform an atomic store release and an atomic notify after the callback returns in order to unblock a concurrent callback deregistration (if one exists).

If there are no stop-callbacks registered (the best case scenario) then the `request_stop()` operation will perform:

- atomic load relaxed (read 'locked' flag = 0, 'stop-requested' flag = 0)
- atomic compare-exchange weak acq-rel (set 'locked' flag = 1, 'stop-requested' flag = 1)
- read list of callbacks and find empty list
- store release (set 'locked' flag = 0)

If there are stop-callbacks registered then the `request_stop()` operation will perform the following (assuming no contention):

- atomic load relaxed (read 'locked' flag = 0, 'stop-requested' flag = 0)
- atomic compare-exchange weak acq rel (set 'locked' flag = 1, 'stop-requested' flag = 1)
- while callback list is non-empty
 - remove next callback
 - atomic store release (set 'locked' flag = 0)
 - atomic notify
 - invoke callback
 - atomic load relaxed (read 'locked' flag = 0)
 - atomic compare-exchange weak acq rel (set 'locked' flag = 1)
- atomic store release (set 'locked' flag = 0)

If there is contention on the stop-source lock, then each of the lock-acquisition operations will enter a spin-loop with atomic-wait backoff.

5.1.2. Cost of single_inplace_stop_token

1. Sizes of data-structure

The `single_inplace_stop_source` structure only needs to store a single atomic pointer plus a thread-id.

On 64-bit platforms, this structure will typically be 16 bytes in size. On some platforms this is the same as `inplace_stop_source` and on others is 8-bytes smaller.

The `single_inplace_stop_callback` structure needs to store a pointer to the stop-source and also a function-pointer to invoke when the stop-callback is invoked, along with any state that the user's callback object requires, which is often also a single pointer.

On 64-bit platforms this will be 16 bytes plus the size of the user's callback. So in most cases, 24-bytes per stop-callback. Compare this to 56-bytes per stop-callback for the reference implementation of `std::inplace_stop_callback`.

It is worth noting, however, that for `std::inplace_stop_callback` we can have many stop-callback objects for a single `std::inplace_stop_source`. Whereas for `std::single_inplace_stop_callback`, each callback object needs to be associated with a different stop-source object.

Consider, for example, a `when_all()` algorithm that has 10 child operations, where each child registers a single stop-callback in the child operation-state.

An implementation that uses a single `std::inplace_stop_source` in the `when_all()` operation-state would use 24 bytes in the parent operation-state and 56 bytes for a stop-callback in each of the 10x child operation-states - a total of 584 bytes.

An implementation that uses 10x `std::single_inplace_stop_source` objects in the `when_all()` operation state would use 10x 16 bytes in the parent operation state and 24 bytes in each of the 10x child operation-states - a total of 400 bytes.

The size usage can be further reduced by using the proposed `std::finite_inplace_stop_source` which supports N separate stop-tokens, each with their own stop-callback slot in the stop-source. In this case, we could store the `std::thread::id` of the thread requesting stop once and then only require an extra pointer for each callback-slot in the stop-source. i.e. it would be (N + 1) pointers in size instead of 2 * N pointers in size.

In this case, the storage needed for the `when_all()` algorithm with 10x children could be further reduced to $8 + 10 * 8 + 10 * 24 = 328$ bytes, compared to the current implementation in terms of `std::inplace_stop_source` which takes 584 bytes - a saving of 256 bytes.

2. Cost of operations

1. Registering a stop-callback

Registering a stop-callback involves an atomic load-relaxed to see if a stop-request has already been made and if not then a single compare-exchange strong to try to install the stop-callback. The compare-exchange will only fail if there has been a stop-request issued and so it does not need to be performed in a loop.

Thus, if a stop-request has been issued already then the operations are:

- atomic load acquire (reads a state that indicates stop-requested)

If stop-request has not been issued then the operations will be:

- atomic load acquire (reads a state that indicates no stop-request has been made)
- atomic compare-exchange strong release (stores the stop-callback address)

2. Deregistering a stop-callback

Deregistering the stop-callback involves trying to compare-exchange the atomic pointer from pointing to the registered callback back to nullptr. If successful, then the deregistration has won the race and the callback is successfully deregistered. Otherwise, if the compare-exchange fails then this indicates that there was a stop-request which either has or is in the process of invoking the callback.

In this case, the atomic pointer will have been set to the 'stop-requested' value before the callback is invoked and is set to the 'stop-requested-callback-done' value after the callback returns. The deregistration just needs to wait until the atomic pointer value is no longer equal to the 'stop-requested' value, which can only happen when the thread calling `request_stop()` sets it to 'stop-requested-callback-done' value.

We also need to handle the case where the callback is being deregistered from within the stop-callback, as in this case we don't want to block until the stop-callback returns as this would deadlock. We detect this case by comparing the current thread to the thread-id of the thread calling `request_stop()` (which is written to

So, in the case that there has not yet been a stop-request, the cost is:

- 1x successful compare-exchange strong w/ relaxed memory order

In the case that there has been a stop-request and the callback has already finished executing, we have:

- 1x unsuccessful compare-exchange strong w/ acquire memory order

In the case that there has been a stop-request and the callback has not yet finished executing on another thread, we have:

- 1x unsuccessful compare-exchange strong w/ acquire memory order
- relaxed load of thread-id from stop-state
- comparison of this thread-id to `std::this_thread::get_id()`
- `atomic::wait()` on pointer with acquire memory order to wait for value to change from the 'stop-requested' value.

If the deregistration occurs on the same thread as the thread calling `request_stop()` then the operations are the same, we just skip the `atomic::wait()` call.

3. Calling `request_stop()`

The `request_stop()` implementation of `single_inplace_stop_source` tries to atomically compare-exchange the pointer to be the 'stop-requested' value, as long as it does not already have a value that indicates a stop-request has been made.

Since the current value of the atomic pointer can potentially be changed concurrently by another thread registering/deregistering a stop-callback, it needs to perform a compare-exchange in a loop until either the compare-exchange succeeds or it observes that another thread has made a stop-request.

If the compare-exchange is successful, it then inspect the previous value. If there was a stop-callback registered, then it set the thread-id field to the current thread-id and invokes the stop-callback. When the stop-callback invocation returns, it writes the 'stop-requested-callback-done' value to the atomic pointer and notifies any waiting threads.

So if there is no stop-callback registered (and no contention) then we have:

- 1x atomic load acquire - reads null pointer
- 1x successful compare-exchange weak w/ acq-rel memory order - stores 'stop-requested' value

If there is a stop-callback registered (and no contention) then we have:

- atomic load acquire - reads non-null pointer
- 1x successful compare-exchange weak w/ acq-rel memory order - stores 'stop-requested' value
- atomic store relaxed of current thread-id
- invoke callback
- atomic store release - stores 'stop-requested-callback-done' value
- atomic notify

4. A "mostly" lock-free implementation

One important thing to note about the implementation of `single_inplace_stop_source` compared with `inplace_stop_source` is that, for most of the operations, the implementation is now lock-free.

This means that one thread will never be waiting on some other thread to make forward progress in order to complete its operation.

The one exception to this is where a call to `request_stop()` is racing with a concurrent call to deregister a stop-callback on another thread. In this case, the call to deregister the stop-callback may need to block until the invocation of the stop-callback returns to avoid destroying the stop-callback object while it is still being used.

5.1.3. Cost of `finite_inplace_stop_token`

The cost of `finite_inplace_stop_source<N>` is similar to that of an array of `N` `single_inplace_stop_source` objects with a couple of minor differences.

1. Differences in data-structure size

The size of a `finite_inplace_stop_source<N>` can be up to $(N-1)$ pointers smaller than that of an array of `N` `single_inplace_stop_source` objects, reducing the number of cache lines required by the operation-state.

This reduction in storage usage has one potential down-side in that it can potentially increase the amount of false-sharing involved when multiple threads are each trying to register stop-callbacks to the different stop-tokens concurrently. With `single_inplace_stop_source` there would be 4x stop-states sharing a typical 64-byte cache line. Whereas with `finite_inplace_stop_source` the storage for up to 8 stop-state may be grouped together in a single cache-line, increasing the potential for false-sharing.

Whether this is an issue in-practice will depend on the use-case and whether, in practice, there will be multiple threads concurrently trying to register/unregister stop-callbacks.

In most cases it is expected that using less storage will result in overall better performance. However, there are pathological cases where overheads due to false-sharing can be significant. It is worth noting that both approaches can be subject to false-sharing overheads to some degree.

2. Difference in operation-cost

The cost of registering/deregistering stop-callbacks will be largely the same as that of `single_inplace_stop_source`.

The main performance difference will be in terms of the cost of the `request_stop()` operation.

With `finite_inplace_stop_source::request_stop()`, the implementation can do two things more efficiently compared to an array of `single_inplace_stop_source`:

1. It only needs to call `std::this_thread::get_id()` and store in the data-structure once for all stop-tokens, rather than once for each stop-token.
2. Once it has decided the race of which thread called `request_stop()` first using a compare-exchange, deciding the race between `request_stop()` and registration/deregistration of a stop-callback can be done with an unconditional atomic exchange instead of an atomic compare-exchange loop, which at least on some platforms, is slightly more efficient.

For measurements of the performance differences see [Appendix A - Benchmarks](#).

5.2. Performance vs Safety Tradeoff

The proposed `std::single_inplace_stop_token` type adds extra pre-conditions to the construction of `std::single_inplace_stop_callback` objects which are not there for the `std::inplace_stop_callback` type.

This means that users of this stop-token type need to be more careful about its usage to ensure that only a single associated stop-callback exists at a time as violating this pre-condition can lead to undefined behaviour.

It also means that sender-algorithm implementers need to be more careful when forwarding a `get_stop_token` query to multiple child operations.

It is worth noting that in most cases where an algorithm that has multiple child operations with overlapping asynchronous operation lifetimes it will often want to explicitly control the cancellation behaviour. For example, by sending a stop-request to the other child operations when it receives a particular result from one of the children.

This tends to be an inherent part of designing a concurrent algorithm, and implementations that want to control cancellation will tend to need to introduce a new stop-source and register a stop-callback with the parent environment's stop-token that forwards to this new stop-source anyway, thus avoiding the problem of passing a single-callback-stop-token to multiple, concurrent child operations.

Authors of new concurrent sender algorithms tend to need to be aware of lots of the lifetime constraints anyway and implementing them is an advanced use of the sender/receiver framework.

The constraints that this paper proposes to put on usage of stop-tokens in a sender/receiver context should not affect most users, who we expect to be largely composing existing senders. As long as sender-algorithms abide by the restrictions, composing those algorithms together should be transparent.

The benefits to users are that when they use sender-algorithms that take advantage of the single-callback constraints, their code uses less memory (operation-state objects are smaller) and runs more efficiently (uses less CPU-time).

5.3. Usage in task coroutines

While there is not yet a concrete proposal for a task coroutine type that integrates with sender/receiver, something that will need to be considered in such a proposal is what stop-token type the coroutine will provide in the environment connected to awaited senders.

On the one hand, a coroutine can only await a single child operation at a time, and so if the stop-token is only ever propagated to child operations by `co_await` expressions then it seems reasonable to have the task coroutine provide a `std::single_inplace_stop_token` rather than a `std::inplace_stop_token` so that we can take advantage of the better performance.

However, one of the use-cases that is not uncommon in task coroutines is to use the `read_env()` algorithm to obtain the current stop-token from the environment, and then to construct a stop-callback as a local variable in the coroutine.

For example: A coroutine that calls a low-level, cancellable OS API, using the coroutine's stop-token natively

```
void os_operation_start(void(*callback)(int, void*), void* data);
void os_operation_cancel(void* data);

task<int> dequeue() {
    struct state_t {
        async_manual_reset_event event;
        std::optional<int> result;
    };

    state_t state;

    auto on_stop_request = [&] noexcept { os_operation_cancel(&state); };

    auto on_complete = [](int result, void* data) noexcept {
        auto& state = *static_cast<state_t*>(data);
        state.result = result;
        state.event.set();
    };

    std::stoppable_token auto st =
        co_await std::execution::read_env(std::execution::get_stop_token);

    os_operation_start(on_complete, &state);

    {
        // Register a stop-callback that will cancel the os_operation if a
        // stop-request comes in before the operation completes.
        std::stop_callback_for_t<decltype(st), decltype(on_stop_request)> cb{st, on_stop_re

        co_await state.event.wait();
    }

    if (st.stop_requested()) {
        // Complete with 'stopped' result
        co_await std::execution::just_stopped{};
    }

    co_return result.value();
}
```

In this example, the coroutine obtains the current stop-token using the `read_env` algorithm and then constructs a stop-callback associated with that stop-token. Then, while this stop-callback object is still alive, the coroutine then awaits on the `async_manual_reset_event` to suspend the coroutine until the callback passed to `os_operation_start()` is invoked.

However, the coroutine will also need to pass an environment with a stop-token down to all `co_await` expressions so that stop-requests can be transparently propagated through coroutines to child operations. However, the child operation might then go on to register its own stop-callback to that stop-token.

If the task coroutine were to use a `single_inplace_stop_token` for its stop-token then this would run into potential problems with trying to attach multiple stop-callbacks.

Therefore, it's likely that we either need to ban such usage within a coroutine, or we need the coroutine's stop-token type to be chosen to allow multiple stop-callbacks to be attached. e.g. by using `inplace_stop_token` instead.

This will mean that when you try to compose a task object as a sender into algorithms like `when_all` that there will need to be an adapter that adapts between the `finite_inplace_stop_token`, passed by `when_all` in the

environment to the task, and a new `inplace_stop_source` that can produce an `inplace_stop_token` that can propagate through the chain of coroutines.

This is an example of a situation where trying to use a more efficient stop-token type can actually end up hurting performance in cases where you needed an `inplace_stop_token` anyway.

5.4. Do we still need `inplace_stop_token`?

Yes, we still need to keep the `inplace_stop_token` family of types.

While it is still a generally useful facility, there are two main use-cases for it in the facilities targeting C++26.

The first is the use by the yet-to-be-proposed task type mentioned above.

The second is the use by a cancellable `counting_scope` type proposed in [P3149](#).

In this case, a cancellable `counting_scope` may spawn an unbounded number of tasks running within the `async_scope`. If you want to cancel all of the operations in that scope then you need some way to send a stop-request to all of those spawned operations. The easiest way to do that is to have a single `inplace_stop_source` and then to just pass an associated `inplace_stop_token` in the environment passed to the spawned operation.

6. Proposed Wording

Proposed wording will be forthcoming in a future revision of this paper.

7. Appendix A - Benchmarks

This section contains some micro-benchmarks that look at relative performance of different kinds of stop-source data-structures proposed in this paper compared to the existing `inplace_stop_source` type.

As with all micro-benchmarks, the results should be taken with a large grain of salt. I have added some interpretation comments where I thought appropriate.

The source and raw output of the benchmarks can be found at <https://gist.github.com/lewisbaker/d95b3a001650c509570af4968b0d00c5>

Benchmarks were evaluated on an AMD Ryzen 5950X using Clang 19 with compile-flags `-std=c++2c -O2 -stdlib=libc++ -DNDEBUG=1`.

With all of these benchmarks, the operation is performed 100k times per run. e.g. registering + unregistering a single callback 100k times. So, if you divide the time for the benchmark by 100k to get the per-operation times.

And then the run is performed multiple times and statistics gathered on the different runs. For single-threaded benchmarks, the shortest time is reported. For multi-threaded benchmarks, where the results are more variable, the min/max/p50/avg values are all reported to give a better picture of the distribution.

7.1. Register/unregister stop-callbacks single-threaded

This benchmark tests the performance of registering and unregistering a single callback onto a single stop-source 100k times.

Data-structure	Elapsed Time
inplace_stop_source	788us
single_inplace_stop_source	533us

7.2. Call `request_stop()` with no callbacks

This benchmark tests the performance of calling `request_stop()` on various stop-source configurations when there are no associated stop-callbacks. This looks at situations where you might have a `when_all()` of multiple child operations, none of which are actually cancellable and thus none of them register any stop-callbacks, so that we can see the relative performance of different strategies.

For the `inplace_stop_source` this only needs to be run once as it has the same data-structure regardless of what the maximum number of children is.

For both `single_inplace_stop_source` and `finite_inplace_stop_source` we run the test in multiple configurations, evaluating for situations where different numbers of stop-callbacks are supported.

Data-structure	Elapsed Time (us)
inplace_stop_source	445
1x single_inplace_stop_source	306
2x single_inplace_stop_source	588
finite_inplace_stop_source<2>	528
3x single_inplace_stop_source	974
finite_inplace_stop_source<3>	661
10x single_inplace_stop_source	3228
finite_inplace_stop_source<10>	2215

It is worth noting here that, for the case where there are no stop-callbacks registered, the `inplace_stop_source` is actually more efficient for all but the case where there is at most a single stop-callback. In this case, the `request_stop()` method only needs to acquire and release the lock once, so the overhead is fairly low.

In the case where we have a maximum callback count of $N > 1$, we end up needing to perform N separate atomic compare-exchange operations to check each potential stop-callback slot, and so the cost of this operation rises linearly with the maximum callback count.

In all cases for $N > 1$, the `finite_inplace_stop_source` has a slight performance advantage over multiple `single_inplace_stop_source` objects, which I mainly attribute to only having to call `std::this_thread::get_id()` once instead of N times and the fact it can use atomic-exchange operations for the second and subsequent slots instead of compare-exchange.

7.3. Call `request_stop()` with x/y callbacks

With the following benchmarks, we have a stop-source configuration that has a particular maximum number of callbacks that can be registered, and then some number of stop-callbacks registered when a call to `request_stop()` is made.

Here we measure the performance of registering the stop-callbacks, calling `request_stop()` and then deregistering the stop-callbacks 100k times, all from a single thread.

The first group looks at the case where there is only a single stop-callback registered.

Data-structure	# / max	Elapsed Time (us)
<code>inplace_stop_source</code>	1 / *	1353
<code>single_inplace_stop_source</code>	1 / 1	939
<code>2x single_inplace_stop_source</code>	1 / 2	1211
<code>finite_inplace_stop_source<2></code>	1 / 2	1120
<code>3x single_inplace_stop_source</code>	1 / 3	1402
<code>finite_inplace_stop_source<3></code>	1 / 3	1325

In the case where there are potentially multiple children but only a single stop-callback has been registered, the

The next group of results looks at the case where we register more than one stop-callback. In this case we are registering the same lambda multiple times.

Data-structure	# / max	Elapsed Time (us)
<code>inplace_stop_source</code>	2 / *	2765
<code>2x single_inplace_stop_source</code>	2 / 2	1845
<code>finite_inplace_stop_source<2></code>	2 / 2	1780
<code>inplace_stop_source</code>	3 / *	4044
<code>3x single_inplace_stop_source</code>	3 / 3	2697
<code>finite_inplace_stop_source<3></code>	3 / 3	2642
<code>inplace_stop_source</code>	10 / *	12929
<code>10x single_inplace_stop_source</code>	10 / 10	9893
<code>finite_inplace_stop_source<10></code>	10 / 10	8825

Here we can see that as the number of registered stop-callbacks goes up, the benefit of the `single_` and `finite_` stop-source data-structures widens.

This is largely attributed to the relatively high cost of synchronization needed for each stop-callback with the `inplace_stop_source` data-structure - requiring to lock/unlock the structure and maintain the next/prev pointers of the doubly-linked list.

It is worth noting that the CPU branch predictor can have a large impact on the performance of these benchmarks. For example, if, instead of registering the same lambda 10x in the last test, we instead register different lambdas such that when invoking each of the registered stop-callbacks in turn it dispatches to a different function-pointer, the performance can be up to 3x slower (e.g. $\sim 24000\text{us}$ for the `finite_inplace_stop_source<10>` benchmark).

7.4. Register/unregister callbacks from two threads concurrently

In this test, we spin up two threads and have each thread simultaneously try to register and unregister a single stop-callback to a stop-source 100k times.

Each run is synchronized by a spin-barrier that tries to have each thread actively running rather than blocked in an OS synchronization call so that we can better evaluate the effect of contention on the stop-source data-structure from concurrent threads. In this sense, this is trying to evaluate the worst-case scenario of multiple threads continually registering/deregistering callbacks and conflicting with each other.

The times from both threads are added to the set of run-times and then the overall results are compared. i.e. it generates two time samples for each run.

In the case of `inplace_stop_source`, both threads try to register stop-callbacks to the same stop-source object.

In the case of `single_inplace_stop_source`, each thread tries to register its callback to a separate stop-source object. I've split this out into two variants to try to highlight the impact of false-sharing in this scenario. The first result in the table shows the performance if both `single_inplace_stop_source` objects live in the same cache-line. The only difference in the code between the second and third rows of the table is that the third row has aligned the stop-source objects sufficiently to ensure that they are placed in different cache-lines.

In the case of `finite_inplace_stop_source<2>`, both threads are given a reference to the same stop-source object, but each thread registers its stop-callback using a different stop-token. `get_token<0>()` for the first thread, and `get_token<1>()` for the second thread.

Data-structure	Min Time (us)	P50 Time	Avg Time	Max Time
<code>inplace_stop_source</code>	2820	7430	6633	7871
<code>2x single_inplace_stop_source</code>	984	5705	4796	6264
<code>2x single_inplace_stop_source (no false sharing)</code>	533	556	569	956
<code>finite_inplace_stop_source<2></code>	1000	5257	4778	6163

The results here are a lot more variable than the single-threaded benchmarks and so I have included the minimum, maximum, 50% percentile (median) and average measurements to get a better idea of the overall distribution of times.

In all of the results except the "no false sharing" result, the mean is skewed lower than the p50 results by some outliers which happened to get lucky and run fast because the threads happened to be scheduled in such a way to contend far less. So the minimum run-times are perhaps less useful to look at.

If we look, instead, at the p50 and average times then we see that all of the runs except "no false sharing" are typically running much slower than we'd expect from the single-threaded runs.

Much of this slow-down can be attributed to the impact of multiple threads conflicting with each other, trying to atomically modify the same cache line. In the case of `inplace_stop_source` this is a true conflict on the shared state, and in the case of the other data-structures, the conflicts are the result of false-sharing (accessing different atomic objects that live in the same cache-line).

This is evidenced by the fact that the "no false sharing" benchmark exhibiting much less variability and times that are much closer to that of the single-threaded performance.

8. Appendix B - Implementation of `single_inplace_stop_source`

The following code shows a reference-implementation of the classes `single_inplace_stop_source`, `single_inplace_stop_token` and the class template `single_inplace_stop_callback`.

For a full implementation of `finite_inplace_stop_source`, see the source code for the above benchmark.

```
#include <atomic>
#include <cassert>
#include <concepts>
#include <thread>
#include <utility>

namespace std
{
    class single_inplace_stop_token;
    template <typename CB>
    class single_inplace_stop_callback;

    //////////////////////////////////////////////////
    // single_inplace_stop_source
    //

    class single_inplace_stop_source {
    public:
        single_inplace_stop_source() noexcept : state_(no_callback_state()) {}

        bool request_stop() noexcept;
        bool stop_requested() const noexcept;

        single_inplace_stop_token get_token() const noexcept;

    private:
        template <typename CB>
        friend class single_inplace_stop_callback;

        struct callback_base {
            void (*execute)(callback_base* self) noexcept;
        };

        bool try_register_callback(callback_base* cb) const noexcept;
        void deregister_callback(callback_base* cb) const noexcept;

        void* stop_requested_state() const noexcept { return &state_; }
        void* stop_requested_callback_done_state() const noexcept {
            return &thread_requesting_stop_;
        }
        static void* no_callback_state() noexcept { return nullptr; }
        bool is_stop_requested_state(void* state) const noexcept {
            return (state == stop_requested_state()) |
                (state == stop_requested_callback_done_state());
        }

        // nullptr                - no stop-request or stop-callback
        // &state_                 - stop-requested
        // &thread_requesting_stop - stop-requested, callback-done
        // other                   - pointer to callback_base
        mutable atomic<void*> state_;
        mutable atomic<thread::id> thread_requesting_stop_;
    };

    inline bool single_inplace_stop_source::stop_requested() const noexcept {
        void* state = state_.load(std::memory_order_acquire);
        return is_stop_requested_state(state);
    }
}
```

```

inline bool single_inplace_stop_source::request_stop() noexcept {
    void* old_state = state_.load(std::memory_order_relaxed);
    do {
        if (old_state == stop_requested_state() ||
            old_state == stop_requested_callback_done_state()) {
            return false;
        }
    } while (!state_.compare_exchange_weak(old_state, stop_requested_state(),
                                           memory_order_acq_rel,
                                           memory_order_relaxed));

    if (old_state != no_callback_state()) {
        auto* callback = static_cast<callback_base*>(old_state);
        thread_requesting_stop_.store(this_thread::get_id(),
                                       memory_order_relaxed);

        callback->execute(callback);

        state_.store(stop_requested_callback_done_state(), memory_order_release);
        state_.notify_one();
    }

    return true;
}

inline bool single_inplace_stop_source::try_register_callback(
    callback_base * base) const noexcept {
    void* old_state = state_.load(memory_order_acquire);
    if (old_state == stop_requested_state() ||
        old_state == stop_requested_callback_done_state()) {
        return false;
    }

    assert(old_state == no_callback_state());

    if (state_.compare_exchange_strong(old_state, static_cast<void*>(base),
                                       memory_order_release,
                                       memory_order_acquire)) {
        // Successfully registered callback.
        return true;
    }

    // Stop request arrived while we were trying to register
    assert(old_state == stop_requested_state());

    return false;
}

inline void single_inplace_stop_source::deregister_callback(
    callback_base * base) const noexcept {
    // Initially assume that the callback has not been invoked and that the
    // state still points to the registered callback_base structure.
    void* old_state = static_cast<void*>(base);
    if (state_.compare_exchange_strong(old_state, no_callback_state(),
                                       memory_order_relaxed,
                                       memory_order_acquire)) {
        // Successfully deregistered the callback before it could be invoked.
        return;
    }

    // Otherwise, a call to request_stop() is invoking the callback.
    if (old_state == stop_requested_state()) {
        // Callback not finished executing yet.
        if (thread_requesting_stop_.load(std::memory_order_relaxed) ==

```

```

        std::this_thread::get_id()) {
        // Deregistering from the same thread that is invoking the callback.
        // Either the invocation of the callback has completed and the thread
        // has gone on to do other things (in which case it's safe to destroy)
        // or we are still in the middle of executing the callback (in which
        // case we can't block as it would cause a deadlock).
        return;
    }

    // Otherwise, callback is being called from another thread.
    // Wait for callback to finish (state changes from stop_requested_state()
    // to stop_requested_callback_done_state()).
    state_.wait(old_state, memory_order_acquire);
}

}

////////////////////////////////////
// single_inplace_stop_token
//

class single_inplace_stop_token {
public:
    template <typename CB>
    using callback_type = single_inplace_stop_callback<CB>;

    single_inplace_stop_token() noexcept : source_(nullptr) {}

    bool stop_possible() noexcept { return source_ != nullptr; }

    bool stop_requested() noexcept {
        return source_ != nullptr && source_>stop_requested();
    }

private:
    friend single_inplace_stop_source;
    template <typename CB>
    friend class single_inplace_stop_callback;

    explicit single_inplace_stop_token(
        const single_inplace_stop_source* source) noexcept
        : source_(source) {}

    const single_inplace_stop_source* source_;
};

inline single_inplace_stop_token single_inplace_stop_source::get_token()
    const noexcept {
    return single_inplace_stop_token{this};
}

////////////////////////////////////
// single_inplace_stop_callback
//

template <typename CB>
struct single_inplace_stop_callback
    : private single_inplace_stop_source::callback_base {
public:
    template <typename Init>
    requires std::constructible_from<CB, Init>
    single_inplace_stop_callback(
        single_inplace_stop_token st,
        Init&& init) noexcept(is_nothrow_constructible_v<CB, Init>)
        : source_(st.source_), callback_(std::forward<Init>(init)) {
        this->execute = &execute_impl;
    }

```

```

        if (source_ != nullptr) {
            if (!source_>try_register_callback(this)) {
                source_ = nullptr;
                execute_impl(this);
            }
        }
    }

    ~single_inplace_stop_callback() {
        if (source_ != nullptr) {
            source_>deregister_callback(this);
        }
    }

    single_inplace_stop_callback(single_inplace_stop_callback&&) = delete;
    single_inplace_stop_callback(const single_inplace_stop_callback&) = delete;
    single_inplace_stop_callback& operator=(single_inplace_stop_callback&&) =
        delete;
    single_inplace_stop_callback& operator=(
        const single_inplace_stop_callback&) = delete;

private:
    static void execute_impl(
        single_inplace_stop_source::callback_base* base) noexcept {
        auto& self = *static_cast<single_inplace_stop_callback*>(base);
        std::move(self.callback_());
    }

    const single_inplace_stop_source* source_;
    [[no_unique_address]] CB callback_;
};

template <typename CB>
single_inplace_stop_callback(single_inplace_stop_token,
                             CB) -> single_inplace_stop_callback<CB>;
}

```